

S3 Tables vs 自管 Iceberg

compaction / 无 compaction 三组对比 • SF200 与 SF2000 双规模 • 含压缩时间对比

测试设计：唯一变量 = compaction

同一 EMR 集群 / 同 Spark 资源 (22×4×18g) / 同源 Parquet / 行数校验一致

A S3 Tables

- 托管 Iceberg catalog
- 自动 compaction
- 零运维

B 自管 + 手动压缩

- Hadoop catalog
- 手动 rewrite_data_files
- 基准 = 1.00

C 自管 无压缩

- Hadoop catalog
- 保持 8MB 小文件
- 不做 compaction

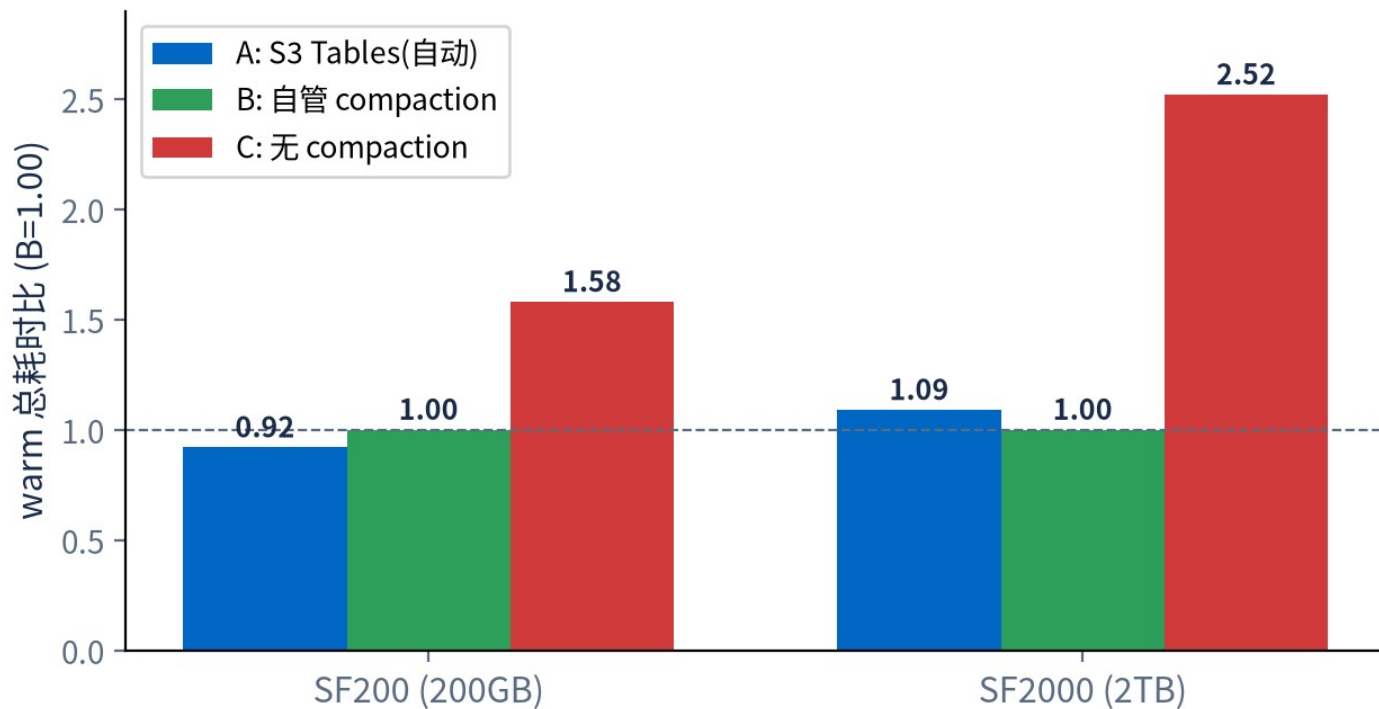
对齐要点

- 两个规模：SF200(200GB) 与 SF2000(2TB)；B/C 用完全相同的 8MB 小文件写入，只对 B 压缩
- 主指标 = warm 均值 (第 2、3 轮平均，剔除冷启动)；查询集 tpcds_2_4 共 103 SQL × 3 轮，0 报错
- 行数三组校验一致：SF2000 store_sales=55 亿 / catalog_sales=28.7 亿 (恰为 SF200 的 10×)

查询性能：compaction 是决定性的

无 compaction 从 SF200 慢 58% → SF2000 慢 152%(2.5×), 规模越大越糟

103 SQL warm 总耗时对比 (越低越快)



warm 总耗时 (秒)

SF200: A=451.8 B=489.4 C=770.6

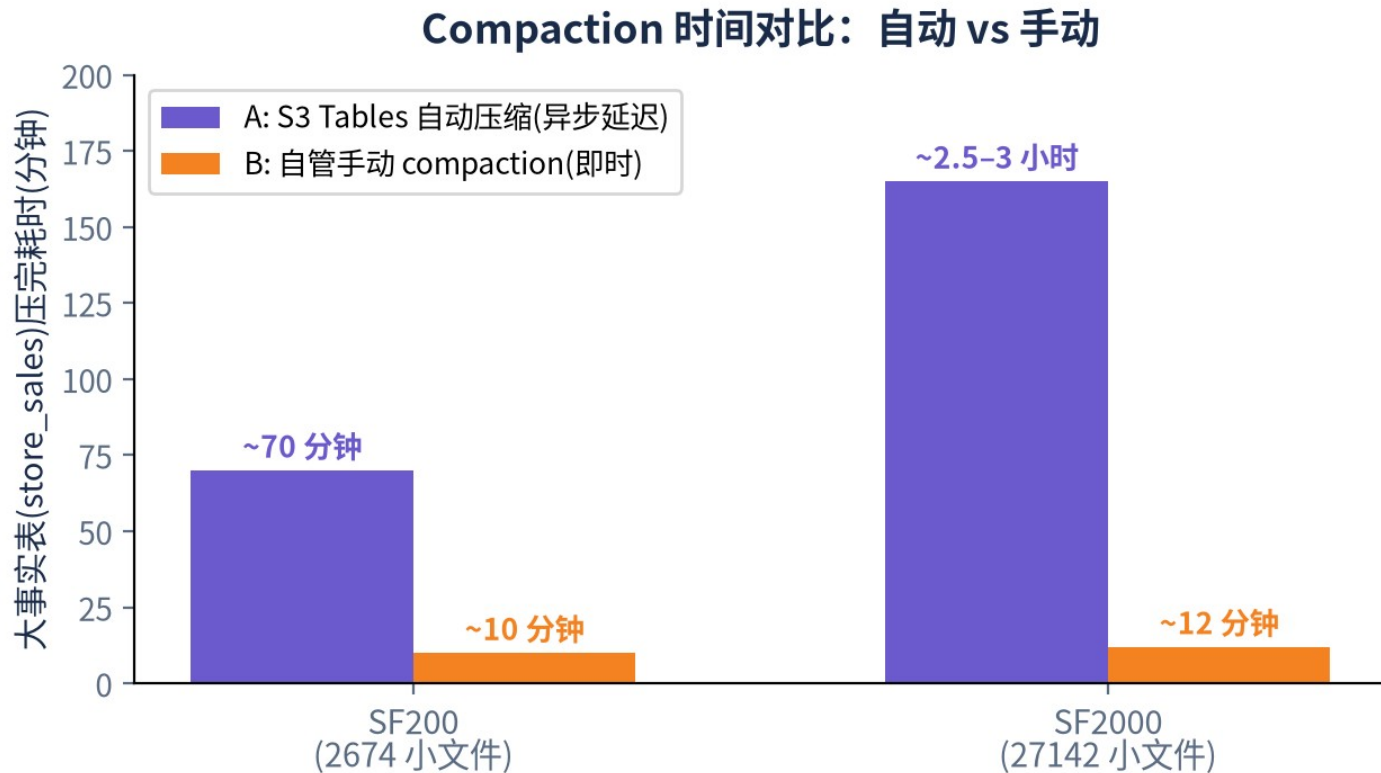
SF2000: A=2256.7 B=2065.2 C=5195.7

关键发现

- C 无压缩 :SF2000 下 94/103 条查询比 B 慢 >20%
- 扫描密集型 (q14a/b、q75、q23) 慢 2.6–3.8×
- 单查询最多多耗 +129 秒 (q14a)
- C 多轮不提速——小文件开销是结构性的, 缓存救不了

⚠️ Compaction 时间对比：自动 vs 手动

最重要运维发现：S3 Tables 自动压缩是异步的，延迟随规模显著变长



大表 (store_sales) 压完耗时

A: S3 Tables 自动 (异步)

SF200 → ~70 分钟

SF2000 → ~2.5-3 小时

B: 自管手动 (即时按需)

SF200 → ~10 分钟

SF2000 → ~12 分钟 (27142→389)

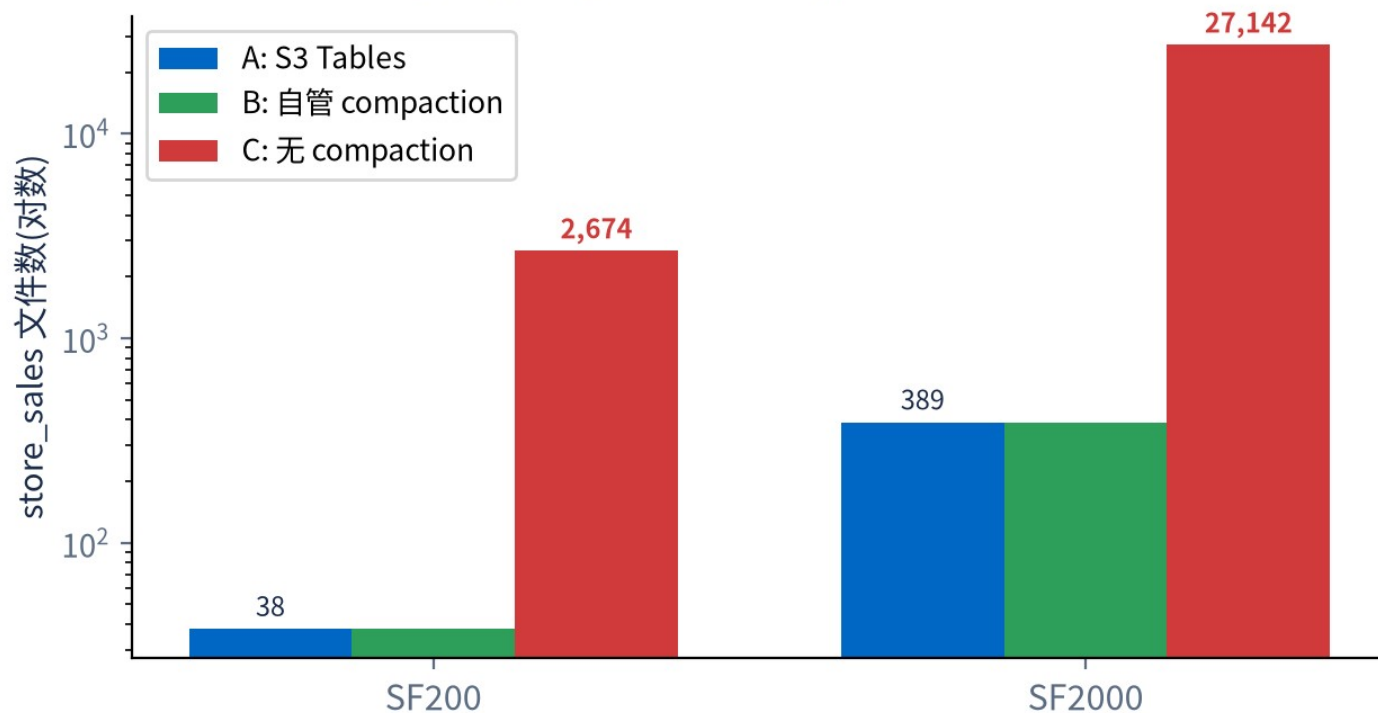
取舍

自动 = 零运维，但新数据有 "未压窗口" 且随规模变长；手动 = 运维换即时可控

小文件问题：无 compaction ≈ 70 倍文件

文件数暴增是查询变慢的根因—— planning/list 开销结构性放大

小文件数对比：无 compaction $\approx 70\times$



store_sales 文件数

SF200: A=38 B=38 C=2674

SF2000: A=389 B=389 C=27142

结论

- A 与 B 大表最终文件数完全一致 (都收敛 512MB target)
- 托管自动压缩与手动压缩查询性能等价 (warm 差异仅 8-9%, 无系统性优劣)
- inventory 表 :SF200 时 S3 Tables 保守不压, SF2000 越阈值才压——自动策略按体量判定

选型建议与结论

三组核心结论 + 场景选型

场景	推荐
不想运维 compaction/snapshot, 能接受新数据 " 未压窗口 "(分钟 ~ 小时 , 随规模增长)	S3 Tables (A)
需要新写入数据即时最优性能 , 或对 compaction 时机 / 策略要求精细可控	自管 + 手动 (B)
任何生产场景	务必做 compaction

三大结论

- ① **Compaction 决定性**
规模越大越关键 ,2TB 无压慢 2.5×
- ② **托管≈手动**
查询性能等价 , 差异仅 8-9%
- ③ **自动压缩有异步延迟**
且随规模变长 (70min→3h)